

Sentiment Analysis of Customer Feedback using BERT

UCSC Extension Course: AISV.X401.(21) Summer 2025

Student: Luis Lopez-Echeto Bordon

Date: August 27, 2025

1. Project Overview

The goal of this project was to develop a machine learning model that automatically classifies short customer feedback as **positive** or **negative**. By leveraging modern natural language processing (NLP) techniques, specifically the transformer-based **BERT** model, we aimed to automate sentiment tracking and reduce the burden of manual review.

This solution has potential applications in customer service centers, social media monitoring, and product review analysis. It can help organizations proactively identify satisfaction trends or dissatisfaction signals and take timely action.

2. Methodology

2.1 Tools and Libraries Used

- Python 3.11
- PyTorch 2.6.0
- Hugging Face Transformers
- scikit-learn (for evaluation)
- pandas
- VSCode with Jupyter Extension

2.2 Dataset Overview

- **Source:** Kaggle – Customer Feedback Dataset
- **Format:** CSV with 7 fields per row:
 - Text , Sentiment , Source , Date/Time , User ID , Location , Confidence Score
- **Size:** 96 feedback entries
- **Classes:** Positive (53), Negative (43)

Preprocessing Steps

Due to the dataset's irregular CSV formatting (with embedded quotes and special characters), manual parsing was required:

- Removed surrounding quotes from each line
- Parsed fields using Python's `csv.reader`
- Built a clean pandas DataFrame
- Cleaned double quotes and stripped whitespace from `Text`
- Mapped `Sentiment` to binary labels:
 - **Positive** → 1
 - **Negative** → 0

2.3 Data Splitting

The dataset was split into:

- **80% Training Set**
- **20% Validation Set**

Using `train_test_split()` from scikit-learn with a fixed `random_state` for reproducibility.

3. Model Design and Training

3.1 Model Architecture

- **Base Model:** `bert-base-uncased` from Hugging Face
- **Fine-Tuning Objective:** Binary sequence classification (`num_labels=2`)
- **Classifier Head:** Linear layer on top of pooled BERT output

3.2 Tokenization and Dataset Preparation

- Used BERT's tokenizer to encode input text
- Applied padding and truncation to ensure consistent input length
- Created a custom PyTorch `Dataset` for the Trainer API

3.3 Training Setup

- **Trainer API** from Hugging Face Transformers
- Trained for **3 epochs**
- Disabled logging with Weights & Biases to keep the environment clean
- GPU acceleration used for faster training

3.4 Model Evaluation

Evaluation was performed on the validation set using `Trainer.evaluate()` .

```
{'eval_loss': 0.0492, 'eval_runtime': 0.068, 'eval_samples_per_second': 293.27}
```

4. Results and Analysis

4.1 Model Performance

- **Training Loss** (final): ~0.26
- **Evaluation Loss**: ~0.049
- **Accuracy Estimate**: >90% (based on evaluation loss and sample testing)

Although traditional metrics like accuracy, precision, and recall weren't explicitly printed, manual testing showed strong predictive capability.

4.2 Inference Example

```
text = "Quick response by the team but the solution didn't work"  
Sentiment: Negative
```

The model correctly predicted that the comment carried a negative sentiment despite the mixed tone.

5. Conclusions

This project successfully fine-tuned a BERT model to classify customer feedback into binary sentiment categories. The pipeline—from raw CSV parsing to prediction—demonstrates how deep learning and NLP can solve practical business problems.

Key Outcomes

- Fully functional text classification model using BERT
 - Cleaned and preprocessed real-world dataset
 - Achieved high performance with limited training data
 - Deployed simple prediction function for inference
-

6. Limitations and Future Work

Limitations

- Small dataset (96 entries) may limit generalization
- Only binary classification (Neutral class excluded)
- Model training and evaluation were not extensive due to dataset size

Future Enhancements

- Expand dataset with more labeled examples
 - Add support for **neutral** sentiment (3-class classification)
 - Deploy model as a web API or chatbot service
 - Use evaluation metrics like F1-score, confusion matrix, and ROC curve for better validation
-

7. GitHub Repository

All source code, notebooks, and data processing scripts are available here:

 [GitHub Project Link](#)

8. Appendix: Sample Data Preview

Text	Sentiment	Label
I love this product!	Positive	1
The service was terrible.	Negative	0
I'm so disappointed with their customer support.	Negative	0
Just had the best meal of my life!	Positive	1
The product arrived damaged . Very disappointed.	Negative	0